Makemore

Part 1 — The Bigram Character-Level Language Model

A complete, cell-by-cell study handbook for the first video in Andrej Karpathy's *makemore* series — “The spelled-out intro to language modeling: building makemore.”

Compiled by: ***Shreyas Pradeepkumar Khandale*** · *GitHub:* ***sherurox***

Companion to: `makemore.ipynb`

What this handbook is

This is a self-contained study companion to your `makemore.ipynb` notebook. It walks the notebook from top to bottom, explaining what each cell does, why it does it, and how it fits into the bigger picture. The goal is that you can read this document alone — on the train, before an interview, the night before extending the project to MLPs — and reconstruct the entire bigram model in your head.

Where useful, I've added Karpathy's own framing from the lecture, the PyTorch mechanics you'll trip over (broadcasting, `keepdims`, `multinomial`, `requires_grad`), and the math behind negative log likelihood. Code blocks are pulled directly from your notebook.

How to use this handbook

- **First pass:** read straight through. Don't run anything; just build the mental model.
- **Second pass:** open the notebook side-by-side and run each cell as you re-read its section here.
- **Revision pass:** read only the **callout boxes** and the **Summary** chapter. They contain the high-signal nuggets you'll be asked about in interviews.
- **For the GitHub README:** Chapter 1 and the Summary make a clean “what this repo is about” section.

Contents

1. The goal: what is makemore?

makemore is a tiny project by Andrej Karpathy that takes a list of things and learns to make *more* of them. The dataset we use is `names.txt` — roughly 32,000 real human names scraped from a government website. Train makemore on this list and it will start generating name-like strings that were not in the training set. Some look like real names. Some are Lord-of-the-Rings villains. All of them are statistically plausible English names.

Underneath, makemore is a **character-level language model**. That means it treats each name as a sequence of characters and tries to predict the next character given the previous one(s). The whole Karpathy makemore series builds the same kind of model — a next-character predictor — using progressively more sophisticated architectures, all the way up to a GPT-2-equivalent Transformer.

Why a character-level model?

Two reasons. First, characters are a small vocabulary (26 letters + 1 special token = 27 tokens) so everything fits in your head and runs on a CPU. Second, every architectural idea — counting, MLPs, convolutions, RNNs, attention — works at the character level. Once you understand it here, scaling up to words, BPE tokens, or full GPT is just bigger matrices.

> **The one-line summary**
> makemore takes a list of strings, learns the statistical structure of the characters in those strings, and then samples new strings from that learned distribution. Part 1 of the series does this with the simplest possible model — a bigram — and shows that two completely different methods (counting tables and gradient descent) arrive at the exact same model.

2. The full series roadmap

The notebook's first markdown cell lays out the seven models we will build across the series. Each one is the same task — predict the next character — with a more powerful architecture:

#	Model	Reference paper
1	Bigram (count-based & neural)	— (this notebook)
2	MLP	Bengio et al. 2003
3	CNN	DeepMind WaveNet 2016
4	RNN	Mikolov et al. 2010
5	LSTM	Graves et al. 2014
6	GRU	Kyunghyun Cho et al. 2014
7	Transformer	Vaswani et al. 2017

Part 1 — this notebook — covers item #1. Crucially, it builds the bigram model **twice**: once by literally counting bigrams in a 27×27 matrix, then again as a one-layer neural network trained by gradient descent. The punchline of the lecture is that those two approaches converge to *the exact same matrix*. That's not a coincidence — it's the whole reason this video is the first one in the series.

3. Loading the dataset — names.txt

The first four cells set up and inspect the data.

Cell 1 — Load all the names into a Python list

```
words = open('names.txt', 'r').read().splitlines()
```

`read()` slurps the whole file into one giant string. `splitlines()` then breaks it on newlines, giving us a list of strings, one per name. We chose `splitlines()` over `split('\n')` because `splitlines()` handles Windows-style `\r\n` line endings without leaving stray carriage returns.

Cells 2–4 — Sanity-check the data

```
words[:10]            # ['emma', 'olivia', 'ava', 'isabella', 'sophia', ...]
len(words)            # 32033 — about 32k names
min(len(w) for w in words)   # 2 — shortest name
max(len(w) for w in words)   # 15 — longest name
```

Always start by knowing the shape of your data. Length 2 to length 15 means we'll see names like *li* and *jo* alongside *genesis* and *kimberly*. The list looks sorted by frequency, with the most common names first.

> **Why this matters**
> Every name is not one training example — it's many. *isabella* tells us: *i* is a likely first letter, *s* is likely after *i*, *a* is likely after *is*, … and the word is likely to *end* after *isabella*. That last bit — the "word ends here" signal — is information we'll need to explicitly encode.

4. The bigram concept

A **bigram** is just a pair of consecutive characters. The bigram language model predicts the next character based *only* on the single previous character. It is the dumbest non-trivial language model you can build, and Karpathy starts here because everything that comes later — MLPs, Transformers — is the same machinery wrapped around a more powerful predictor.

For a name like *emma*, the bigrams are: `(., e), (e, m), (m, m), (m, a), (a, .)`. Notice the `.` tokens at both ends — those are special markers for "start of word" and "end of word." Without them, the model can't learn which letters tend to start names or end them.

Two special tokens or one?
Karpathy initially used <S> for start and <E> for end, giving a 28×28 matrix. But then he realized the start token can never appear as the *second* character of a bigram, and the end token can never appear as the *first* — so a whole row and a whole column are wasted. The fix is to use a single token `.` for both, shrinking the matrix to 27×27. This is the form the notebook ends up with.

5. Counting bigrams with a Python dictionary

Before reaching for PyTorch, we prototype the idea with a plain Python dict.

Cell — Build the bigram counts dictionary

```
b = {}
for w in words[:3]:
    chs = ['<S>'] + list(w) + ['<E>']
    for ch1, ch2 in zip(chs, chs[1:]):
        bigram = (ch1, ch2)
        b[bigram] = b.get(bigram, 0) + 1
        print(ch1, ch2)
```

The key trick here is `zip(chs, chs[1:])`. This pairs each character with the *next* one. `zip` stops as soon as the shorter iterator is exhausted, which is exactly the "sliding window of size 2" behaviour we want.

`b.get(bigram, 0) + 1` is the Pythonic way to increment a key that might not exist yet — get the current count or 0, add 1, store it back. (You could also use `collections.Counter` or `defaultdict(int)`, but `dict.get` is the most explicit.)

Cell — Sort the dictionary by count, descending

```
sorted(b.items(), key=lambda kv: -kv[1])
```

`b.items()` returns `(key, value)` tuples. By default `sorted` would sort by the key (the bigram string), which is alphabetical and not useful. We override that with `key=lambda kv: -kv[1]` — sort by the *negative* of the count, which is equivalent to descending order by count. (Equivalent alternative: `reverse=True` and `key=lambda kv: kv[1]`.)

Run this on the full dataset and you'll see *(a, n)*, *(a, e)*, *(n, a)* at the top — *n* almost always follows *a* in English names. Rare bigrams like *(q, r)* appear once each.

6. Switching to a 2-D tensor — the N matrix

A dict is fine for prototyping, but for sampling and gradient descent we want a real array. We move to PyTorch tensors.

Cell — Import torch and create the empty count matrix

```
import torch
N = torch.zeros((27, 27), dtype=torch.int32)
```

`torch.zeros((27, 27), dtype=torch.int32)` creates a 27×27 tensor filled with zeros, of 32-bit integer type. We use `int32` because we're storing *counts* — they're whole numbers, and the default `float32` would waste memory and be semantically misleading.

Why 27? 26 lowercase letters plus one special start/end token, the `.`.

7. The character lookup tables: stoi and itos

PyTorch tensors can only be indexed by integers, but our bigrams are pairs of strings. We need a string-to-integer mapping (and its inverse).

Cell — Build stoi and itos

```
chars = sorted(list(set(''.join(words))))
stoi = {s: i+1 for i, s in enumerate(chars)}
stoi['.'] = 0
itos = {i: s for s, i in stoi.items()}
```

Reading this line by line:

- `''.join(words)` concatenates every name into one massive string.
- `set(...)` throws out duplicates, leaving the 26 unique lowercase letters used in the dataset.
- `sorted(list(...))` gives a stable, alphabetical ordering: `['a', 'b', 'c', …, 'z']`.
- `{s: i+1 for i, s in enumerate(chars)}` maps `'a'` → `1`, `'b'` → `2`, …, `'z'` → `26`. The +1 leaves slot 0 free.
- `stoi['.'] = 0` reserves index 0 for the special token. This is purely an aesthetic choice — Karpathy says he just finds it neater to have the special token at position 0.
- `itos` is the inverse mapping, built by flipping `stoi`. We'll use it when sampling — once the model gives us an integer, we need to turn it back into a character.

Cell — Populate the count matrix

```
for w in words:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs, chs[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        N[ix1, ix2] += 1
```

This is the dictionary-counting loop from earlier, but writing into `N` instead of `b`. After it runs, `N[ix1, ix2]` is the number of times character `ix1` was immediately followed by character `ix2` across all 32,000 names.

Row 0 of `N` tells us which characters start names. Column 0 tells us which characters end names. The interior 26×26 block tells us the inner transitions.

8. Visualizing the count matrix

Cell — Plot the matrix with matplotlib

```
import matplotlib.pyplot as plt
%matplotlib inline

plt.figure(figsize=(16, 16))
plt.imshow(N, cmap='Blues')
for i in range(27):
    for j in range(27):
        chstr = itos[i] + itos[j]
        plt.text(j, i, chstr, ha='center', va='bottom', color='gray')
        plt.text(j, i, N[i, j].item(), ha='center', va='top', color='gray')
plt.axis('off');
```

This is the famous “heatmap of bigram counts” that appears in every makemore blog post and tweet. Each cell shows the bigram string (top) and its count (bottom), with darker blue = higher count. You can *see* the structure of English names jump out: the row for `.` has dark cells for common starting letters (*a, j, k, m, s*); the column for `.` has dark cells for common ending letters (*a, e, n, h*).

> **Why `.item()`?**
> When you index a PyTorch tensor with two integers, you get back a 0-D tensor, not a Python int. `matplotlib`'s text rendering wants a plain number, so we call `.item()` to extract the scalar Python value from the tensor.

9. From counts to probabilities (one row at a time)

To sample a character, we need a *probability distribution*, not raw counts. Probabilities are non-negative and sum to 1. We get there by dividing each row by its sum.

Cell — Normalize the first row

```
N[0]                # raw counts for what follows '.' (i.e., name starts)
p = N[0].float()    # cast to float because we're about to divide
p = p / p.sum()     # now p is a probability distribution
```

Three things are happening: (1) we grab row 0, which is the count of characters that start a name; (2) we cast to `float` because integer division would round to zero; (3) we divide by the sum so the row sums to 1.0.

p is now a 27-vector of probabilities. `p[1]` is P(first letter = 'a'). `p[13]` is P(first letter = 'm'). And so on.

10. torch.multinomial and deterministic sampling

Cell — Seed a generator and sample once

```
g = torch.Generator().manual_seed(2147483647)
ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
itos[ix]
```

`torch.multinomial` takes a probability vector and returns indices sampled according to that distribution. If p says “a” is 14% likely and “j” is 8% likely, then over many draws, 14% of samples will be index 1 (a) and 8% will be index 10 (j).

- `num_samples=1` — draw one sample at a time.
- `replacement=True` — put each drawn index back so it can be drawn again. For our use case (sampling characters) this is required: of course “a” can be drawn twice. Confusingly, the default is `False`, so you have to set it explicitly.
- `generator=g` — uses our seeded PRNG, so the result is deterministic. Run the same cell on your laptop and you'll get the same integer Karpathy got.
- `.item()` — pulls the scalar out of the returned 1-element tensor.

> **Why deterministic generators matter**
> Karpathy seeds g with `2147483647` (which is $2^{31} - 1$, the largest signed 32-bit integer — a stable, recognisable number) so the lecture is reproducible. Your samples will be identical to his. When you write reproducible research code, you'll do the same thing: pick a seed once, document it, and pass it explicitly to every stochastic operation.

11. Sampling from the trained bigram model

Now we sample full names by repeatedly drawing the next character from the row corresponding to the current character, until we draw the end token `.` (index 0).

Cell — The trained sampler

```
g = torch.Generator().manual_seed(2147483647)

for i in range(20):
    out = []
    ix = 0                      # start at '.', i.e., name-start
    while True:
        p = N[ix].float()
        p = p / p.sum()
        ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
        out.append(itos[ix])
        if ix == 0:             # if we sampled '.', the name ends
            break
    print(''.join(out))
```

The output looks like:

```
mor.
axx.
minaymoryles.
kondlaisah.
anchshizarie.
...
```

These names look bad, but they look *like names*. They have plausible vowel/consonant patterns. Compare this to the untrained model, which uses a uniform distribution over all 27 characters:

Cell — The untrained (uniform) sampler — for comparison

```
g = torch.Generator().manual_seed(2147483647)
```

```
for i in range(20):
    out = []
    ix = 0
    while True:
        p = torch.ones(27) / 27.0      # uniform: every character equally likely
        ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
        out.append(itos[ix])
        if ix == 0:
            break
    print(''.join(out))
```

The uniform sampler produces strings like *cexze.* and *krmnbnsmwqz.* — total garbage. The contrast tells us that even the bigram model — the simplest possible — has learned *something* real about English name structure.

12. Efficiency: building the full P matrix with broadcasting

Inside the sampling loop, we recomputed `p = N[ix].float(); p = p / p.sum()` every single iteration. That's wasteful — those probabilities never change. We should compute the full 27×27 probability matrix `P` once and look up rows from it.

Cell — Precompute P

```
P = N.float()
P /= P.sum(1, keepdims=True)
```

`N.float()` casts to float. `P.sum(1, keepdims=True)` sums along dimension 1 (the columns), giving us a 27×1 column vector where row *i* contains the total count emitted by character *i*. Then `P /=` that column vector divides every row by its own sum — which is exactly normalization.

> **Why `keepdims=True`?**
> Without it, `P.sum(1)` returns a 1-D tensor of shape `(27,)` — the dimension we summed over has been "squeezed out." With `keepdims=True` we get shape `(27, 1)`, preserving the rank. This matters because of broadcasting (next chapter): `(27, 27) / (27, 1)` correctly divides each row by its sum, but `(27, 27) / (27,)` would silently divide each *column* by a different number — a catastrophic, hard-to-detect bug.

13. Broadcasting rules — the bug that almost happened

PyTorch (and NumPy) automatically "stretch" tensors of different shapes to make an operation work — this is called **broadcasting**. The rules, walked from the right, are:

- Align the shapes from the right.
- Two dimensions are compatible if they are equal, or if one of them is 1.

- A missing dimension on the left is treated as 1.
- If at any axis the dimensions don't match and neither is 1, the operation errors out.

In our case:

```
P            shape (27, 27)
P.sum(1, keepdims=True)   shape (27,  1)

Aligned from right:
             27 27
             27  1

Dimension 1: 27 vs 1 → broadcast the 1 across 27 ✓
Dimension 0: 27 vs 27 → match ✓
```

So the `(27, 1)` column gets stretched into a `(27, 27)` where every column is a copy of the sum vector. The division then happens element-wise. The result: every row of `P` is divided by its own sum. Correct.

> **What would have gone wrong without `keepdims`**
> Without `keepdims`, the sum has shape `(27,)`. Aligned from the right: `(27, 27)` vs `(27,)` → becomes `(27, 27)` vs `(1, 27)`. PyTorch would then stretch the 1 across rows — meaning every **column** would be divided by the same number, not every row. The probabilities would be wrong, and the model would silently produce nonsense. Bugs like this are the single most common cause of broken PyTorch code.

Cell — Sample using the precomputed P

```
g = torch.Generator().manual_seed(2147483647)

for i in range(20):
    out = []
    ix = 0
    while True:
        p = P[ix]                    # just look up the row
        ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
        out.append(itos[ix])
        if ix == 0:
            break
    print(''.join(out))
```

Same output as before, but no per-step arithmetic. Pure lookup.

14. Evaluating the model: log-likelihood and NLL

We have a model. We have samples. How do we say *numerically* how good the model is? We need a single number that summarises model quality — a **loss**.

The intuition

If the model is good, it should assign high probability to the bigrams that actually appear in the training data. If we multiply together the probabilities the model assigns to every real bigram, we get the **likelihood** of the dataset under the model. A good model makes this product large.

But this product is the product of ~228,000 numbers each less than 1 — it will underflow to zero in any computer. So we take the **log**. Log turns products into sums, and is monotonic (bigger likelihood → bigger log-likelihood), so optimizing one is the same as optimizing the other.

Karpathy's chain of equivalences:

- **Goal:** maximize the likelihood of the data under the model (this is what statistical modelling means).
- Equivalent to maximizing the *log*-likelihood (because log is monotonic).
- Equivalent to minimizing the *negative* log-likelihood (because flipping the sign flips max → min).
- Equivalent to minimizing the *average* negative log-likelihood (dividing by N is just a scale; doesn't change the argmin).

So our loss is the **average negative log-likelihood**, abbreviated **NLL** or **avg-NLL**.

Cell — Compute the loss on the first three names

```
log_likelihood = 0.0
n = 0
for w in words[:3]:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs, chs[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        logprob = torch.log(P[ix1, ix2])
        log_likelihood += logprob
        n += 1
        print(f'{ch1}{ch2}: {P[ix1, ix2]} {logprob:.4f}')

print(f'{log_likelihood=}')
nll = -log_likelihood
print(f'{nll=}')
print(f'{nll/n}')
```

We loop through every bigram in the first 3 names, look up the probability the model assigns to it, take the log, and accumulate. Then we negate (NLL) and divide by the count (avg-NLL).

> **What the numbers mean**
> For a uniform model, every bigram has probability 1/27 ≈ 0.037. `log(1/27) ≈ -3.30`, so avg-NLL would be ≈ 3.30. Our trained bigram model gets avg-NLL ≈ 2.45 on the full dataset — meaningfully better than uniform. The theoretical minimum is 0 (every bigram given probability 1.0), which is impossible. More complex models (MLPs, Transformers) push this number down further. **Lower NLL = better model.**

Personalized example — your name

In your notebook you evaluated the loss on the string *shreyas*:

```
for w in ["shreyas"]:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs, chs[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        logprob = torch.log(P[ix1, ix2])
        log_likelihood += logprob
        n += 1
        print(f'{ch1}{ch2}: {P[ix1, ix2]} {logprob:.4f}')
```

This prints, bigram by bigram, how likely the model thinks each pair is. For *shreyas*, expect the bigram *hr* to be quite unlikely (so its `logprob` will be a big negative number), pulling the avg-NLL up. The model has never seen many Indian-origin names, so its loss on *shreyas* will be worse than its loss on *emma*.

15. The 'jq' problem — when probability is zero

Cell — Evaluate on a sequence the model considers impossible

```
for w in ["jq"]:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs, chs[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        logprob = torch.log(P[ix1, ix2])
        log_likelihood += logprob
        n += 1
        print(f'{ch1}{ch2}: {P[ix1, ix2]} {logprob:.4f}')
```

The bigram *jq* never appears anywhere in the training set. So `N[j, q] = 0`, and `P[j, q] = 0`, and `log(0) = -inf`. The NLL blows up to infinity. The model is saying “this is literally impossible” — which is too strong. We just didn't happen to see *jq* in our 32k names; that doesn't mean no human ever named their kid Jqing.

An infinite loss is also a problem mathematically — it makes the loss undefined for any test set containing this bigram, and it would make gradient descent (later) explode.

16. Model smoothing

Cell — Add 1 to every count before normalizing

```
P = (N + 1).float()
P /= P.sum(1, keepdims=True)
```

The fix is embarrassingly simple: add 1 (or any positive constant) to every count before computing probabilities. This is called **Laplace smoothing** or just **add-k smoothing**. It guarantees that no probability is ever exactly zero, so no log is ever `-inf`.

The trade-off is a knob:

- Add 0 → original model, but with `-inf` losses on rare bigrams.

- Add 1 → small bump for unseen bigrams; barely changes the trained probabilities. Safe default.
- Add 1,000,000 → swamps the real counts; the model becomes nearly uniform.

Karpathy uses +1. We'll see in Chapter 26 that this smoothing has a deep equivalent in the neural net framework: L2 regularization on the weights does the exact same thing.

17. Bridge: from counting to a neural network

At this point we have a respectable bigram model built by counting. We can sample from it, evaluate it, and smooth it. So why do anything more?

Two reasons:

- **Scaling.** Counting works because a bigram only has 27×27 = 729 possible inputs. If we want a trigram, that's 27^3 = 19,683. For a 10-gram, $27^{10} \approx 2\times10^{14}$ — astronomically too many entries to count, and most would never appear in any finite training set.
- **Flexibility.** The counting approach is a closed-form solution to one specific problem. A neural network is a general-purpose function approximator: same setup, swap the architecture, and you can train an MLP, an RNN, or a Transformer with no other changes to the pipeline.

So we now re-derive the same bigram model — but as a neural network trained by gradient descent. The punchline is that we'll end up with the same probabilities. But the *machinery* is what we need.

> **Karpathy's framing**
> "The neural network is going to receive a single character as input, have some weights, and output a probability distribution over the next character. We can evaluate any setting of the parameters using the NLL loss. We'll use gradient-based optimization to tune the weights so the network correctly predicts the probabilities of the next character. None of this changes as we work all the way up to Transformers — only the forward pass gets more complex."

18. Building the (x, y) training set

Supervised learning needs inputs and labels. For the bigram model: input = previous character; label = next character. Both are integers.

Cell — Build xs and ys for the first word

```
xs, ys = [], []

for w in words[:1]:                # just 'emma' for now
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs, chs[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        print(ch1, ch2)
        xs.append(ix1)
        ys.append(ix2)
```

```
xs = torch.tensor(xs)
ys = torch.tensor(ys)
```

For the word *emma*, we get five training examples — the bigrams `(., e), (e, m), (m, m), (m, a), (a, .)`. As integer indices: `xs = [0, 5, 13, 13, 1]` and `ys = [5, 13, 13, 1, 0]`.

The model's job: when you put 0 in, it should output a distribution that gives 5 high probability; when you put 5 in, it should give 13 high probability; and so on.

> `torch.tensor` **vs** `torch.Tensor`
> Lowercase `torch.tensor(...)` infers the dtype from the data (here, int64). Uppercase `torch.Tensor(...)` always returns a float32 tensor. Always use the lowercase one unless you have a specific reason not to. The docs are not great on this; the convention is community knowledge.

19. One-hot encoding

Neural networks multiply their inputs by weights. You can't meaningfully multiply an arbitrary integer by a weight — the integer 5 doesn't mean "5× as much as integer 1." The integer is a *categorical* label. We need a representation that says "this is class 5, and nothing else."

That representation is the **one-hot vector**: a 27-dimensional vector that's all zeros except for a single 1 at the index corresponding to the class.

Cell — One-hot encode the inputs

```
import torch.nn.functional as F
xenc = F.one_hot(xs, num_classes=27).float()
xenc
```

`F.one_hot` takes integer indices and the size of the vocabulary. For `xs = [0, 5, 13, 13, 1]` with 27 classes, it returns a 5×27 tensor where row *i* is the one-hot vector for `xs[i]`.

We immediately cast with `.float()`. By default `F.one_hot` returns a 64-bit integer tensor (it inherits the dtype of the input), but neural-net inputs must be float for gradients to flow.

20. The single linear layer — W as 27 neurons

Cell — Initialize the weight matrix

```
g = torch.Generator().manual_seed(2147483647)
W = torch.randn((27, 27), generator=g, requires_grad=True)
```

`torch.randn` samples from a standard normal distribution (mean 0, std 1). We create a 27×27 matrix because we have 27 input dimensions (one-hot vector size) and 27 neurons in our layer (one per possible next character). Every neuron has 27 incoming weights — one for each input dimension.

`requires_grad=True` tells PyTorch: "please record all operations involving this tensor in the autograd graph so I can compute its gradient later." This is the one knob that makes PyTorch start tracking the computation graph for backprop.

Why 27 neurons?

Because we want 27 numbers out — one for each possible next character. The output of the linear layer will be interpreted (after softmax) as a probability distribution over the 27 characters.

Cell — Forward through the layer

```
(xenc @ W)[3, 13]
```

`@` is matrix multiplication in PyTorch. `xenc` is 5×27, `W` is 27×27, so `xenc @ W` is 5×27. The element at `[3, 13]` is the activation of the 13th neuron on the 3rd input example. Karpathy calls this the "firing rate" of that neuron — borrowed from neuroscience terminology.

> **The hidden duality**
> One-hot encoding makes matrix multiplication mathematically equivalent to a lookup. If `xenc[i]` is a one-hot vector with a 1 at position *j*, then `(xenc @ W)[i] = W[j]` — literally the *j*-th row of W. So this single linear layer is doing exactly what our count-matrix lookup did, but with learnable parameters. This equivalence is the key insight that makes the rest of the lecture click.

21. Softmax: logits → counts → probabilities

The output `xenc @ W` contains arbitrary real numbers — some positive, some negative. But we want probabilities: non-negative numbers that sum to 1. We get there in two steps:

- **Exponentiate** the output, element-wise. The exp function maps the entire real line to the positive reals: negative inputs become numbers between 0 and 1, positive inputs become numbers greater than 1. So `logits.exp()` is always non-negative.
- **Normalize** by dividing each row by its sum. Now every row sums to 1.

Together, these two steps are the **softmax** function. We interpret the raw network output as *log-counts* (also called **logits**); exponentiating gives counts; normalizing gives probabilities. The entire pipeline mirrors the count-based approach, but with everything differentiable.

Cell — The full forward pass

```
xenc = F.one_hot(xs, num_classes=27).float()    # one-hot input
logits = xenc @ W                               # log-counts
counts = logits.exp()                           # equivalent to N
probs = counts / counts.sum(1, keepdims=True)   # row-normalize → P
loss = -probs[torch.arange(5), ys].log().mean() # avg-NLL
```

The mental mapping:

- `logits` ↔ log of the count matrix N
- `counts` ↔ the count matrix N itself
- `probs` ↔ the probability matrix P
- `W` ends up being log(N) at convergence — same model, different parameterization.

22. The vectorized NLL loss

Look closely at this line:

```
loss = -probs[torch.arange(5), ys].log().mean()
```

Unpacking it from the inside out:

- `torch.arange(5)` → `[0, 1, 2, 3, 4]`, the indices of the 5 examples.
- `probs[torch.arange(5), ys]` → advanced indexing. For example 0, grab `probs[0, ys[0]]`; for example 1, grab `probs[1, ys[1]]`; etc. Result: a 5-vector containing the probability the network assigned to the *correct* next character in each example.
- `.log()` → take log of each of those probabilities. Numbers between 0 and 1 give negative logs.
- `.mean()` → average them.
- `-` → flip the sign, because we want to minimize the negative of the (already negative) log probabilities.

This is exactly the avg-NLL we computed manually in Chapter 14, but in one vectorized line — no Python for-loop, just tensor operations. This is the form gradient-based training uses.

23. The backward pass: requires_grad and loss.backward()

Cell — Backprop

```
W.grad = None       # reset (None is faster than zero-ing)
loss.backward()     # populate W.grad with d(loss)/d(W)
W.grad              # inspect: tensor of shape (27, 27)
```

When we did `W = torch.randn(..., requires_grad=True)`, PyTorch started tracking every operation involving W. Each operation knows its own derivative formula. When we call `loss.backward()`, PyTorch walks the computation graph from `loss` back to `W`, applying the chain rule, and fills in `W.grad` — a tensor of the same shape as W telling us how much the loss would change per unit change in each weight.

If `W.grad[i, j]` is positive, increasing `W[i, j]` increases the loss. So we want to **decrease** that weight. If it's negative, we want to increase the weight. The general rule: nudge weights in the *opposite* direction of the gradient.

> **Why `W.grad = None` and not `W.grad.zero_()`?**
> PyTorch accumulates gradients on `.grad` by default — every `loss.backward()` *adds* to whatever is there. So before each backward pass we have to clear it. Setting to `None` is slightly faster than zeroing (no actual memory write — PyTorch interprets `None` as "no gradient"). Both work; `None` is the modern idiom. This is why your training loop will always call `optimizer.zero_grad()` or the equivalent before each backward pass.

24. The parameter update

Cell — Take one gradient step

```
W.data += -0.1 * W.grad
```

This is the simplest possible optimizer: vanilla SGD with learning rate 0.1.

- `W.data` — directly modify the underlying tensor data, bypassing autograd tracking. We don't want this update itself to be part of the computation graph; we just want to mutate the weights in place.
- `-0.1` — the *learning rate*, times the negative direction of the gradient. The minus sign is what makes this *descent* (we move opposite to the gradient).

Print the loss after the update and you'll see it has gone down (slightly). Repeat the cycle — forward pass, compute loss, zero gradients, backward pass, update — and the loss keeps dropping. This is gradient descent.

25. The full training loop on all 228,000 bigrams

We now scale up: build the training set from *all* words, not just the first one. Reinitialize W. Run gradient descent for 100 iterations.

Cell — Build the full dataset

```
xs, ys = [], []
for w in words:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs, chs[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        xs.append(ix1)
        ys.append(ix2)
xs = torch.tensor(xs)
ys = torch.tensor(ys)
num = xs.nelement()
print('number of examples: ', num)

g = torch.Generator().manual_seed(2147483647)
W = torch.randn((27, 27), generator=g, requires_grad=True)
```

`num` ends up at 228,146 — total bigrams across the dataset. `.nelement()` returns the number of elements in a tensor (here, the same as `len(xs)` since it's 1-D).

Cell — Train for 100 steps

```
for k in range(100):
    # forward pass
    xenc = F.one_hot(xs, num_classes=27).float()
    logits = xenc @ W
    counts = logits.exp()
    probs = counts / counts.sum(1, keepdims=True)
    loss = -probs[torch.arange(num), ys].log().mean() + 0.01 * (W**2).mean()
    print(loss.item())
```

```
    # backward pass
    W.grad = None
    loss.backward()

    # update
    W.data += -50 * W.grad
```

Three things to notice in this training loop:

- **Learning rate is 50**, not 0.1. The bigram problem is convex and well-conditioned; we can afford very large steps. For typical deep networks you'd use 0.001–0.1.
- **The loss has an extra term:** `+ 0.01 * (W**2).mean()`. This is L2 regularization, the deep equivalent of model smoothing (next chapter).
- **No batching, no shuffling.** Every step is full-batch gradient descent on all 228k bigrams. That's only tractable here because the model is tiny.

After 100 iterations, the loss converges to about 2.47 — essentially identical to the loss the count-based model achieved (~2.45 with smoothing). The two approaches end up at the same answer because, fundamentally, they are the same model.

26. Regularization = smoothing (the deep equivalence)

In the counting world, “model smoothing” meant adding +1 to every count, pulling the distribution closer to uniform. More added → more uniform.

In the neural-net world, we add `+ 0.01 * (W**2).mean()` to the loss. This term pushes every weight toward zero. Why does that have the same effect as smoothing?

Trace through what happens when all weights are zero:

- `logits = xenc @ W` → all zeros.
- `counts = logits.exp()` → all ones (because `exp(0) = 1`).
- `probs = counts / counts.sum(1, keepdims=True)` → every row is `[1/27, 1/27, ..., 1/27]` — a uniform distribution.

So *pulling weights toward zero pulls the model's output toward uniform*. That is exactly what smoothing did. The regularization strength (0.01) controls how strong that pull is — bigger coefficient → more uniform → smoother predictions, at the cost of higher training loss.

Why this matters beyond the bigram model

L2 regularization is everywhere in deep learning — under names like “weight decay” in Adam, or the `weight_decay` argument in your optimizer. The mechanism is always the same: add the sum-of-squared-weights to the loss, and the optimizer is incentivized to keep weights small. The concrete interpretation (“makes predictions smoother / more uniform / less confident”) is a useful intuition every time you tune this hyperparameter.

27. Sampling from the trained neural net

Finally, we sample names from the trained network. The procedure is the same as before — start at `.`, repeatedly draw the next character — but instead of looking up `P[ix]`, we run the network's forward pass to produce the distribution.

Cell — Sample from the neural net

```
g = torch.Generator().manual_seed(2147483647)

for i in range(50):
    out = []
    ix = 0
    while True:
        xenc = F.one_hot(torch.tensor([ix]), num_classes=27).float()
        logits = xenc @ W
        counts = logits.exp()
        p = counts / counts.sum(1, keepdims=True)

        ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
        out.append(itos[ix])
        if ix == 0:
            break
    print(''.join(out))
```

The output is essentially identical to the count-based sampler. Same names. Same quality. Same model.

If you compare this loop with the count-based sampler from Chapter 11, the only difference is the four lines computing `p` — instead of `p = P[ix]` we do the full one-hot → matmul → exp → normalize. Everything else is unchanged. And the punchline of the whole lecture: those four lines are mathematically equivalent to `p = P[ix]`, with `W` playing the role of `log(P)`.

28. Summary and key takeaways

What we built

A bigram character-level language model, trained on ~32k human names. The model predicts the next character given the previous one. We built it twice:

- **Counting approach:** count every bigram, normalize each row to get probabilities, sample.

- **Neural-net approach:** one-hot encode the input, multiply by a learnable 27×27 weight matrix, apply softmax to get probabilities, train by minimizing avg-NLL with gradient descent.

Both approaches arrive at the same model — the matrix W ends up being log(P).

The five concepts that matter most

- **1. Negative log likelihood (NLL).** The standard loss for classification / next-token prediction. Take the log of the probability the model assigns to the correct answer, average over examples, negate. Lower = better. The theoretical minimum is 0.
- **2. Softmax.** Maps any vector of real numbers into a probability distribution: exponentiate, then normalize. Used as the final layer of every classifier / language model from here through GPT.
- **3. One-hot encoding.** Categorical inputs (like character indices) must become vectors before being fed to a neural net. A one-hot vector is the simplest such encoding; later we'll see learned embeddings, which are essentially "dense one-hot vectors."
- **4. Broadcasting.** When dividing matrices of different shapes, PyTorch silently stretches the smaller one. Get the alignment wrong and you'll get incorrect results with no error — the most common source of subtle bugs. Always use `keepdims=True` on reduction operations whose result will be broadcast.
- **5. Regularization = smoothing.** Adding a penalty on `(W**2).mean()` to the loss pulls weights toward zero, which pulls the output toward uniform. Identical effect to adding +k to every count in the counting world. This duality reappears throughout deep learning.

What's coming next in the series

Part 2 (MLP, Bengio 2003) keeps everything in this notebook — the dataset prep, the NLL loss, the softmax output, the gradient descent loop — and replaces the single linear layer with a multi-layer perceptron that takes a context of multiple previous characters. The forward pass complexifies; everything else stays the same. This pattern continues all the way to the Transformer.

> **The single thing to remember if you remember nothing else**
> Building a language model = define a forward pass that turns context into logits, run softmax, compute avg-NLL against the true next token, backpropagate, update. Every model in the series — bigram, MLP, RNN, Transformer — uses this same outer scaffold. Only the forward pass changes.

29. Glossary

Bigram. A pair of consecutive items in a sequence. In our case, two consecutive characters.

Character-level language model. A model that predicts the next character given the previous one(s), as opposed to a word- or token-level model.

Logit. The raw output of a neural network layer, before softmax. We interpret logits as log-counts.

Softmax. exp(x) / sum(exp(x)). Turns a vector of arbitrary reals into a probability distribution.

Likelihood. The probability the model assigns to the observed data. For a dataset of bigrams, the product of P(bigram) for every bigram.

Log-likelihood. The log of the likelihood. Sums become products; products become sums. Bigger = better.

Negative log-likelihood (NLL). –log(likelihood). Smaller = better. The standard classification loss.

Average NLL. NLL divided by the number of examples. Lets you compare losses across datasets of different sizes.

Cross-entropy loss. Same thing as avg-NLL when labels are one-hot. `F.cross_entropy` in PyTorch computes `-log(softmax(logits))[correct_class]` in one numerically-stable call.

One-hot vector. A vector that is all zero except for a single 1 at the index of the class.

Broadcasting. The rule by which PyTorch/NumPy automatically stretch tensors of different shapes to make element-wise operations work.

Multinomial sampling. Drawing indices from a categorical probability distribution.

Generator (PRNG). An object that holds the state of a pseudo-random number stream. Seeded with an integer to make results reproducible.

Forward pass. The computation from inputs to loss.

Backward pass. The computation from loss to gradients, via the chain rule. Triggered in PyTorch by `loss.backward()`.

Gradient. Partial derivative of the loss with respect to a parameter. Tells you how the loss would change if you nudged the parameter.

Gradient descent. Iterative optimization: compute the gradient, take a small step in the opposite direction, repeat.

Learning rate. The size of each gradient step. The most-tuned hyperparameter in machine learning.

Regularization. Any technique that constrains the model to be simpler or smoother. L2 regularization penalizes the sum of squared weights.

Model smoothing (add-k / Laplace). Adding a small constant to every count before normalizing, to avoid zero probabilities. The count-based equivalent of L2 regularization.

requires_grad. A flag on a PyTorch tensor that tells autograd to track operations on it for backpropagation.

keepdims. An option on reduction operations (sum, mean, max) that preserves the reduced dimension as size 1 instead of squeezing it out. Crucial for safe broadcasting.

nelement. Method on a PyTorch tensor that returns the total number of elements.

End of Part 1. The next handbook will cover the Bengio 2003 MLP — the same loss, the same training loop, with a context window of multiple previous characters and a learned embedding table replacing the one-hot lookup.